

Day 48 — HTTP with the requests Library

EngiDock DevOps Training | Month 2: Git & Python Scripting

Topic: The requests library makes HTTP calls from Python simple and reliable — APIs, webhooks, and health checks all use it.

Why HTTP and APIs Are Unavoidable in DevOps

Modern infrastructure is held together by HTTP. Every tool you use as a DevOps engineer exposes an API — GitHub for code, AWS for cloud resources, Slack for alerts, PagerDuty for incidents, Grafana for dashboards, Datadog for metrics, Kubernetes for container management. When you write automation, you're not just moving files around. You're talking to services over HTTP.

Think about a realistic deployment pipeline: a webhook hits your CI server, which calls the GitHub API to clone code and update a commit status, calls the Docker Registry API to push an image, sends a Slack message to notify the team, and opens a PagerDuty incident if something fails. Every single one of those steps is an HTTP call. Without the ability to make HTTP calls from Python, your automation stops at the edge of your own server.

The `requests` library is the de facto standard for HTTP in Python. It wraps Python's lower-level `urllib` in a clean, readable API that handles encoding, headers, authentication, redirects, sessions, and error detection without boilerplate. It's installed in virtually every DevOps Python project, every Ansible plugin, every monitoring script, every custom deployment tool. Learning it thoroughly pays dividends immediately.

Today you'll build real patterns that appear in production environments: a Slack alerting function, a retry loop that handles rate limiting, and a health check wrapper. By Day 49, you'll assemble these into an operational health check script that you can actually run against your infrastructure.

Section 1 — Why HTTP and APIs Are Unavoidable in DevOps

Every service you manage or interact with speaks HTTP. The pattern is always the same: you send a request (with a method, URL, headers, and maybe a body), and you receive a response (with a status code, headers, and maybe a body).

Your Script	Remote Service
GET /repos/myorg/myapp/pulls	
Host: api.github.com	
Authorization: Bearer tk_...	
----->	
HTTP/1.1 200 OK	
Content-Type: application/json	
X-RateLimit-Remaining: 4987	
[{"number": 42, "title": "...}"]	
<-----	

APIs your scripts will commonly call:

```
# GitHub – check PR status, create releases, update commit status
GITHUB_API = "https://api.github.com"

# Slack – send deployment notifications, alert on failure
SLACK_API = "https://hooks.slack.com/services/T.../B.../xxx"

# PagerDuty – create/resolve incidents programmatically
PAGERDUTY = "https://api.pagerduty.com"

# AWS (via boto3, which uses HTTP under the hood)
AWS_EC2 = "https://ec2.amazonaws.com"

# Grafana – query dashboards, create annotations for deploys
GRAFANA_API = "https://grafana.company.com/api"

# Internal services – health checks, config endpoints
INTERNAL_API = "https://api.internal.company.com"

# httpbin.org – testing and learning HTTP (returns what you send)
HTTPBIN = "https://httpbin.org"
```

Section 2 — Installing requests and the Response Object

Installation

```
# Inside your virtual environment (you should always be in a venv)
pip install requests

# Verify
python -c "import requests; print(requests.__version__)"

# In a requirements.txt file
echo "requests>=2.28.0" >> requirements.txt
pip install -r requirements.txt
```

The Response Object — Anatomy

Every `requests` call returns a `Response` object. Understanding its attributes is the foundation of everything else.

```
import requests

response = requests.get('https://httpbin.org/get')

# --- Status ---
print(f"Status code: {response.status_code}") # 200, 404, 500, etc.
print(f"Success?: {response.ok}") # True if status < 400
print(f"Reason: {response.reason}") # 'OK', 'Not Found', etc.
```

```

# --- URL ---
print(f"Final URL:      {response.url}")          # After any redirects

# --- Timing ---
print(f"Elapsed:        {response.elapsed}")      # timedelta object
print(f"Elapsed (ms):   {response.elapsed.total_seconds() * 1000:.0f}ms")

# --- Headers ---
print(f"Content-Type:   {response.headers['Content-Type']}")
print(f"Content-Length: {response.headers.get('Content-Length', 'unknown')}")
# headers dict is case-insensitive: headers['content-type'] works too

# --- Body: Three Ways to Read It ---

# As a string (decoded using encoding from headers)
text_body = response.text
print(f"Body (text):    {text_body[:100]}")

# As raw bytes (for binary data: images, files, etc.)
bytes_body = response.content
print(f"Body (bytes):   {len(bytes_body)} bytes")

# As parsed JSON (raises json.JSONDecodeError if not valid JSON)
if 'json' in response.headers.get('Content-Type', ''):
    data = response.json()
    print(f"Body (json):  {data}")

# --- Encoding ---
print(f"Encoding:         {response.encoding}")    # 'utf-8', etc.
# You can override it before calling .text:
response.encoding = 'utf-8'
text = response.text

```

Checking Success Before Parsing

```

import requests

response = requests.get('https://api.example.com/data')

# Pattern 1: Check status code directly
if response.status_code == 200:
    data = response.json()
    print(f"Got {len(data)} items")
else:
    print(f"Request failed: {response.status_code} {response.reason}")

# Pattern 2: Use response.ok (True for status < 400)
if response.ok:
    data = response.json()
else:
    print(f"Failed: {response.status_code}")

```

```
# Pattern 3: raise_for_status() – raises HTTPError on 4xx/5xx
# This is the cleanest pattern for "fail fast"
response = requests.get('https://api.example.com/data')
response.raise_for_status()    # Raises if 4xx or 5xx
data = response.json()        # Only reached if status was 2xx
```

Section 3 — GET Requests

GET is the most common HTTP method. It retrieves data. It should never have side effects — calling the same GET twice should return the same result.

```
import requests

# --- Simple GET ---
response = requests.get('https://httpbin.org/get')
print(f"Status: {response.status_code}")
data = response.json()
print(f"Your IP: {data['origin']}")

# --- GET with Query Parameters ---
# Always use params={} instead of building the URL string manually
# requests handles URL encoding for you

# This:
params = {
    'name': 'devops-script',
    'day': '48',
    'env': 'production',
}
response = requests.get('https://httpbin.org/get', params=params)

# Builds URL: https://httpbin.org/get?name=devops-script&day=48&env=production
print(f"Final URL: {response.url}")
data = response.json()
print(f"Args sent: {data['args']}")

# --- GET with Headers ---
headers = {
    'Accept': 'application/json',
    'User-Agent': 'myapp-deploy-script/1.0',
}
response = requests.get(
    'https://httpbin.org/headers',
    headers=headers
)
data = response.json()
print(f"Headers seen by server: {data['headers']}")

# --- GitHub API Example ---
def get_github_releases(owner: str, repo: str, token: str) -> list:
```

```

"""Get all releases for a GitHub repo."""
url = f"https://api.github.com/repos/{owner}/{repo}/releases"

headers = {
    'Authorization': f"Bearer {token}",
    'Accept': 'application/vnd.github+json',
    'X-GitHub-API-Version': '2022-11-28',
}

response = requests.get(url, headers=headers, timeout=10)

if response.status_code == 401:
    raise ValueError("Invalid GitHub token")
elif response.status_code == 404:
    raise ValueError(f"Repository not found: {owner}/{repo}")

response.raise_for_status() # Handle other errors

releases = response.json()
return [
    {
        'tag': r['tag_name'],
        'name': r['name'],
        'draft': r['draft'],
        'date': r['published_at'],
    }
    for r in releases
]

# --- Paginated GET (Common Pattern) ---
def get_all_pages(base_url: str, headers: dict) -> list:
    """
    Handle pagination for APIs that return 'Link: <next_url>; rel="next"' headers.
    GitHub, GitLab, and many REST APIs paginate results.
    """
    all_items = []
    url = base_url
    page = 1

    while url:
        response = requests.get(url, headers=headers, params={'per_page': 100}, timeout=10)
        response.raise_for_status()

        items = response.json()
        all_items.extend(items)

        # Check if there's a next page (Link header)
        link_header = response.headers.get('Link', '')
        if 'rel="next"' in link_header:
            # Parse: <https://api.github.com/...?page=2>; rel="next"
            for part in link_header.split(','):
                if 'rel="next"' in part:
                    url = part.split(';')[0].strip().strip('<>')

```

```

        break

    else:
        url = None    # No more pages

    page += 1
    if page > 50:     # Safety limit
        print("WARNING: Hit page limit (50 pages)")
        break

    return all_items

```

Section 4 — Response Headers

HTTP headers carry metadata about the response — content type, cache control, rate limits, server information, cookies. You'll use them constantly.

```

import requests

response = requests.get('https://httpbin.org/get')

# --- Reading Headers ---
# response.headers is a case-insensitive dict
content_type = response.headers['Content-Type']    # Raises KeyError if missing
content_type = response.headers.get('Content-Type') # Returns None if missing
content_type = response.headers.get('Content-Type', 'unknown') # With default

print(f"Content-Type: {content_type}")

# All headers (case-insensitive lookup)
print("\nAll response headers:")
for key, value in response.headers.items():
    print(f" {key}: {value}")

# --- Rate Limiting Headers ---
# Most APIs enforce rate limits – when you call too fast, they return 429
# They tell you the limit and when it resets via headers

def check_rate_limit(response) -> dict:
    """Extract rate limit info from GitHub API response headers."""
    h = response.headers

    remaining = int(h.get('X-RateLimit-Remaining', -1))
    limit      = int(h.get('X-RateLimit-Limit', -1))
    reset_ts   = int(h.get('X-RateLimit-Reset', 0))

    from datetime import datetime
    reset_time = datetime.fromtimestamp(reset_ts) if reset_ts else None

    return {
        'remaining': remaining,
        'limit':     limit,
    }

```

```

        'reset_at': reset_time.isoformat() if reset_time else 'unknown',
        'used':      limit - remaining if limit > 0 and remaining >= 0 else -1,
    }

# --- Content-Type: Know What You're Getting ---
def parse_response_body(response):
    """Parse response body based on Content-Type header."""
    content_type = response.headers.get('Content-Type', '')

    if 'application/json' in content_type:
        return response.json()
    elif 'text/' in content_type:
        return response.text
    elif 'application/octet-stream' in content_type:
        return response.content # bytes
    else:
        # Unknown type - return bytes, let caller decide
        return response.content

# --- Useful Headers to Know ---
COMMON_RESPONSE_HEADERS = """
Content-Type           What format the body is in (application/json, text/html, etc.)
Content-Length         How many bytes in the body
Content-Encoding       If the body is compressed (gzip, deflate, br)
X-RateLimit-Limit      Max requests per time window
X-RateLimit-Remaining  Requests left in current window
X-RateLimit-Reset      Unix timestamp when window resets
X-Request-Id           Unique ID for this request (for debugging)
Retry-After            How many seconds to wait after a 429 (rate limit)
WWW-Authenticate      What auth method the server requires
Location              Redirect target (for 3xx responses)
ETag                  Cache validation token
Cache-Control          How long to cache this response
"""

# --- Retry-After Header: Handling 429 Correctly ---
def handle_rate_limit(response) -> int:
    """Return how many seconds to wait if we hit a rate limit."""
    if response.status_code != 429:
        return 0

    # Retry-After can be seconds or an HTTP date
    retry_after = response.headers.get('Retry-After', '60')

    try:
        return int(retry_after) # Seconds
    except ValueError:
        return 60 # Default to 60 seconds if parsing fails

```

Section 5 — POST Requests — json= vs data= vs files=

POST sends data to a server — creating a resource, triggering an action, sending a webhook payload. How you send the body depends on what the server expects.

```
import requests

# --- json= : Send JSON (use this for APIs, webhooks) ---
# Sets Content-Type: application/json automatically
# Serializes the dict to JSON for you

payload = {
    'app_name': 'myapp',
    'version': '2.1.0',
    'deployed_by': 'ci-pipeline',
}

response = requests.post('https://httpbin.org/post', json=payload)
data = response.json()

print(f"What server received as JSON: {data['json']}")
print(f"Content-Type sent: {data['headers']['Content-Type']}")
# Content-Type: application/json

# --- data= : Send form data (use for HTML forms, some older APIs) ---
# Sets Content-Type: application/x-www-form-urlencoded
# Think: the same format as an HTML form submission

form_payload = {
    'username': 'alice',
    'action': 'deploy',
}

response = requests.post('https://httpbin.org/post', data=form_payload)
data = response.json()

print(f"\nWhat server received as form: {data['form']}")
print(f"Content-Type sent: {data['headers']['Content-Type']}")
# Content-Type: application/x-www-form-urlencoded

# --- files= : Send file upload (multipart/form-data) ---
# Sets Content-Type: multipart/form-data automatically
# Use for file uploads (log files, artifacts, etc.)

# Upload a file from disk
with open('/tmp/deploy.log', 'rb') as f:
    response = requests.post(
        'https://httpbin.org/post',
        files={'file': (f'deploy.log', f, 'text/plain')}
    )

# Upload file content from memory (no file on disk needed)
```



```

log_content = "Deploy started at 2026-05-19 10:00:00\nBuild: SUCCESS\n"
response = requests.post(
    'https://httpbin.org/post',
    files={'logfile': ('deploy.log', log_content.encode(), 'text/plain')},
    data={'environment': 'production', 'version': '2.1.0'} # Additional form fields
)

# --- Comparison: What Each Sends ---
print("""
Method                Content-Type Header                When to Use
-----
json=payload          application/json                   REST APIs, webhooks, Slack
data=payload          application/x-www-form-urlencoded  HTML forms, some legacy APIs
files=files            multipart/form-data               File uploads
data=raw_string       (whatever you set)               Raw body with custom Content-Type
""")

# --- Sending Raw Body with Custom Content-Type ---
import yaml # pip install pyyaml

yaml_content = """
app: myapp
version: 2.1.0
environment: production
"""

response = requests.post(
    'https://httpbin.org/post',
    data=yaml_content.encode('utf-8'),
    headers={'Content-Type': 'application/yaml'}
)

```

Section 6 — PUT, PATCH, DELETE

REST APIs use HTTP methods to indicate the type of operation. Understanding when to use each is important for writing correct API clients.

```

import requests

# --- PUT: Replace an entire resource (idempotent) ---
# "Replace the whole thing with this"
# Calling PUT twice with the same body should produce the same result
# Use for: updating a config file, replacing a deployment spec

new_config = {
    'replicas': 3,
    'image': 'myapp:2.1.0',
    'environment': 'production',
    'cpu_limit': '500m',
    'memory': '256Mi',
}

```

```

response = requests.put(
    'https://api.example.com/deployments/myapp',
    json=new_config,
    headers={'Authorization': 'Bearer token123'}
)

if response.status_code == 200:
    print("Deployment config updated")
elif response.status_code == 404:
    print("Deployment not found – use POST to create it first")

# --- PATCH: Update part of a resource ---
# "Change just these fields, leave the rest alone"
# Use for: updating a single field, partial config changes

patch_payload = {
    'replicas': 5,    # Only change replica count
}

response = requests.patch(
    'https://api.example.com/deployments/myapp',
    json=patch_payload,
    headers={'Authorization': 'Bearer token123'}
)

# --- DELETE: Remove a resource ---
# Use for: deleting a deployment, removing a webhook, cleaning up

response = requests.delete(
    'https://api.example.com/deployments/old-myapp-v1',
    headers={'Authorization': 'Bearer token123'}
)

if response.status_code == 204:
    print("Resource deleted (204 No Content is normal for DELETE)")
elif response.status_code == 404:
    print("Resource already gone (idempotent – this is fine)")

```

--- REST Convention Summary ---

REST_CONVENTIONS = """

Method	Endpoint	Meaning
GET	/deployments	List all deployments
GET	/deployments/myapp	Get one deployment
POST	/deployments	Create a new deployment
PUT	/deployments/myapp	Replace entire deployment config
PATCH	/deployments/myapp	Update specific fields only
DELETE	/deployments/myapp	Delete the deployment

Status Code Conventions:

200 OK	GET/PUT/PATCH succeeded with a response body
201 Created	POST succeeded and created a new resource

204 No Content	DELETE or PUT succeeded, no body to return
400 Bad Request	Your request payload was malformed
401 Unauthorized	Missing or invalid authentication
403 Forbidden	Authenticated but not allowed to do this
404 Not Found	Resource doesn't exist
409 Conflict	Resource already exists (on POST) or version conflict
422 Unprocessable	Valid JSON but logically invalid (missing required field)
429 Too Many Req	Rate limited – slow down
500 Server Error	API bug – not your fault, but handle it
503 Unavailable	API is down – retry later

```

"""
print(REST_CONVENTIONS)

```

Section 7 — Authentication Patterns

Almost every production API requires authentication. There are several standard patterns — you need to know all of them.

```

import os
import requests
from requests.auth import HTTPBasicAuth

# RULE: Never hardcode credentials. Always read from environment variables.

# --- Pattern 1: Bearer Token (most common for modern APIs) ---
# Used by: GitHub, Slack, Grafana, PagerDuty, most REST APIs

token = os.environ.get('API_TOKEN')
if not token:
    raise ValueError("API_TOKEN environment variable is required")

response = requests.get(
    'https://api.github.com/user',
    headers={
        'Authorization': f"Bearer {token}",
        'Accept': 'application/json',
    },
    timeout=10
)

# --- Pattern 2: HTTP Basic Auth ---
# Used by: older APIs, some internal services, Jenkins
# Sends Base64(username:password) in the Authorization header

username = os.environ.get('API_USERNAME')
password = os.environ.get('API_PASSWORD')

response = requests.get(
    'https://jenkins.company.com/api/json',
    auth=HTTPBasicAuth(username, password), # requests handles encoding
    timeout=10
)

```

```

)

# Equivalent shorthand:
response = requests.get(
    'https://jenkins.company.com/api/json',
    auth=(username, password),  # Tuple is auto-converted to BasicAuth
    timeout=10
)

# --- Pattern 3: API Key in Header ---
# Used by: Datadog, some internal APIs

api_key = os.environ.get('DATADOG_API_KEY')
app_key = os.environ.get('DATADOG_APP_KEY')

response = requests.get(
    'https://api.datadoghq.com/api/v1/hosts',
    headers={
        'DD-API-KEY': api_key,
        'DD-APPLICATION-KEY': app_key,
    },
    timeout=10
)

# --- Pattern 4: API Key in Query Parameter ---
# Used by: some older APIs, AWS pre-signed URLs
# Less secure (key appears in server logs and browser history)

api_key = os.environ.get('WEATHER_API_KEY')

response = requests.get(
    'https://api.openweathermap.org/data/2.5/weather',
    params={
        'q': 'London',
        'appid': api_key,  # In query param, not header
    },
    timeout=10
)

# --- Production-Grade Auth Helper ---

import sys

def get_auth_headers(service: str) -> dict:
    """
    Get authentication headers for a given service.
    All credentials come from environment variables.
    """
    service = service.lower()

    if service == 'github':
        token = os.environ.get('GITHUB_TOKEN')
        if not token:

```

```

        print("ERROR: GITHUB_TOKEN not set", file=sys.stderr)
        sys.exit(1)
    return {
        'Authorization': f"Bearer {token}",
        'Accept': 'application/vnd.github+json',
    }

elif service == 'slack':
    token = os.environ.get('SLACK_BOT_TOKEN')
    if not token:
        print("ERROR: SLACK_BOT_TOKEN not set", file=sys.stderr)
        sys.exit(1)
    return {
        'Authorization': f"Bearer {token}",
        'Content-Type': 'application/json',
    }

elif service == 'pagerduty':
    token = os.environ.get('PAGERDUTY_TOKEN')
    if not token:
        print("ERROR: PAGERDUTY_TOKEN not set", file=sys.stderr)
        sys.exit(1)
    return {
        'Authorization': f"Token token={token}",
        'Accept': 'application/vnd.pagerduty+json;version=2',
    }

else:
    raise ValueError(f"Unknown service: {service}")

```

Section 8 — Error Handling — The Exception Hierarchy

HTTP calls can fail in many ways: the server returns an error status, the network is unreachable, the connection times out, there are too many redirects. You need to handle each case explicitly.

```

import requests
from requests.exceptions import (
    HTTPError,
    ConnectionError,
    Timeout,
    TooManyRedirects,
    RequestException,
)

# --- raise_for_status() ---
# The simplest way to turn HTTP error codes into exceptions
# Raises requests.exceptions.HTTPError for 4xx and 5xx responses

response = requests.get('https://httpbin.org/status/404')
try:
    response.raise_for_status()

```

```

except requests.exceptions.HTTPError as e:
    print(f"HTTP error: {e}")
    print(f"Status code: {response.status_code}")
    print(f"Response body: {response.text[:200]}")

# --- The Exception Hierarchy ---
"""
RequestException          ← Catch-all for all requests errors
├─ HTTPError              ← 4xx/5xx responses (after raise_for_status)
├─ ConnectionError        ← Network-level failure (DNS, refused, reset)
│   ├─ ProxyError         ← Proxy config problem
│   └─ SSLError           ← TLS/SSL certificate error
├─ Timeout                ← Request exceeded timeout
│   ├─ ConnectTimeout     ← Couldn't establish connection in time
│   └─ ReadTimeout        ← Connected but server stopped sending data
├─ TooManyRedirects       ← Redirect loop
└─ URLRequired            ← Invalid URL
"""

# --- Complete Error Handling Pattern ---

def make_request(url: str, method: str = 'GET', **kwargs) -> dict:
    """
    Make an HTTP request with comprehensive error handling.
    Returns: {'success': bool, 'data': ..., 'error': str, 'status_code': int}
    """
    try:
        response = requests.request(method, url, timeout=(3, 10), **kwargs)

        # Check for HTTP errors
        response.raise_for_status()

        # Parse response
        try:
            data = response.json()
        except ValueError:
            data = response.text

        return {
            'success': True,
            'data': data,
            'status_code': response.status_code,
            'error': None,
        }

    except requests.exceptions.HTTPError as e:
        # Server returned 4xx or 5xx
        return {
            'success': False,
            'data': None,
            'status_code': e.response.status_code if e.response else 0,
            'error': f"HTTP {e.response.status_code}: {e.response.reason}" if
e.response else str(e),

```

```

    }

except requests.exceptions.ConnectionError as e:
    # Network unreachable, DNS failed, connection refused
    return {
        'success': False,
        'data': None,
        'status_code': 0,
        'error': f"Connection error: {str(e)[:100]}",
    }

except requests.exceptions.Timeout:
    # Request took too long
    return {
        'success': False,
        'data': None,
        'status_code': 0,
        'error': f"Request timed out",
    }

except requests.exceptions.TooManyRedirects:
    return {
        'success': False,
        'data': None,
        'status_code': 0,
        'error': "Too many redirects – check the URL",
    }

except requests.exceptions.RequestException as e:
    # Catch-all for anything else
    return {
        'success': False,
        'data': None,
        'status_code': 0,
        'error': f"Request failed: {str(e)[:100]}",
    }

```

Usage

```

result = make_request('https://httpbin.org/get')
if result['success']:
    print(f"Success! Data: {result['data']}")
else:
    print(f"Failed: {result['error']}")

result = make_request('https://httpbin.org/status/500')
print(f"500 result: success={result['success']}, error={result['error']}")

result = make_request('https://httpbin.org/status/429')
print(f"429 result: success={result['success']}, error={result['error']}")

```

Section 9 — Timeouts — Every Request Needs One

This is one of the most common mistakes in production scripts: making HTTP requests without timeouts. A single hung request can freeze your entire deployment pipeline.

```
import requests

# --- WRONG: No timeout - this can hang forever ---
# response = requests.get('https://api.example.com/data') # Dangerous

# --- RIGHT: Always set a timeout ---
response = requests.get('https://httpbin.org/get', timeout=10)

# --- The Two Timeouts (tuple syntax) ---
# timeout=(connect_timeout, read_timeout)
#   connect_timeout: seconds to wait for initial connection to be established
#   read_timeout:    seconds to wait for the SERVER to send a response
# These are independent and both matter

response = requests.get(
    'https://httpbin.org/get',
    timeout=(3, 10) # 3 seconds to connect, 10 seconds to read response
)

# --- What "No Timeout" Means in Practice ---
"""
Scenario: Your deploy script calls an internal API to update a deployment record.
The API server is running, but the database it queries is frozen.
The API accepts your connection but never sends a response.

Without timeout: Your script hangs at that line. Indefinitely.
Your CI pipeline sits at "Updating deployment record..." for hours.
The build never finishes. Engineers are confused. The alarm goes off.

With timeout=(3, 10):
After 10 seconds of silence from the server, requests raises Timeout.
Your script catches it, logs the error, and exits with code 1.
The CI pipeline marks the build failed. An engineer investigates.
Problem identified in 10 seconds instead of hours.
"""

# --- Choosing Timeout Values ---
"""
connect_timeout: How long to wait for the TCP handshake.
    3 seconds is standard. A legitimate server responds in <100ms.
    If you're connecting across continents, use 5-10 seconds.

read_timeout: How long to wait for the server to return data.
    Depends on the operation:
        Health check:      2-5 seconds
        Simple REST call:  5-10 seconds
        Search/query:      15-30 seconds
    """
```



```

File upload:      60-300 seconds (per chunk)
Long-running job: set timeout to expected job duration + buffer
"""

TIMEOUT_GUIDELINES = {
    'health_check': (2, 5),
    'simple_api': (3, 10),
    'database_query': (3, 30),
    'file_upload': (5, 120),
    'build_trigger': (3, 60),
}

# --- Handling Timeout Exceptions ---
import sys

def call_with_timeout(url: str, timeout: tuple = (3, 10)) -> dict | None:
    """Call a URL with a timeout. Return None on timeout."""
    try:
        response = requests.get(url, timeout=timeout)
        response.raise_for_status()
        return response.json()

    except requests.exceptions.ConnectTimeout:
        print(f"ERROR: Could not connect to {url} within {timeout[0]}s", file=sys.stderr)
        print(f"        Is the service running? Is the network reachable?", file=sys.stderr)
        return None

    except requests.exceptions.ReadTimeout:
        print(f"ERROR: {url} connected but didn't respond within {timeout[1]}s",
file=sys.stderr)
        print(f"        The service may be overloaded or the operation is taking too long.",
file=sys.stderr)
        return None

    except requests.exceptions.Timeout:
        # Catches both ConnectTimeout and ReadTimeout
        print(f"ERROR: Request to {url} timed out", file=sys.stderr)
        return None

result = call_with_timeout('https://httpbin.org/delay/2', timeout=(3, 10))
print(f"Result: {result is not None}")

# This will timeout:
result = call_with_timeout('https://httpbin.org/delay/15', timeout=(3, 5))
print(f"Result (should be None): {result}")

```

Section 10 — requests.Session — Persistent Connections

When you make multiple calls to the same API, a `Session` object is more efficient. It reuses the underlying TCP connection, shares default headers and auth across all calls, and handles cookies automatically.

```

import requests
import os

# --- Without Session: Each call opens a new connection ---
# Slow for many calls to the same API
token = os.environ.get('GITHUB_TOKEN', 'fake-token')
base_url = 'https://api.github.com'

response1 = requests.get(f"{base_url}/user", headers={'Authorization': f"Bearer {token}"})
response2 = requests.get(f"{base_url}/user/repos", headers={'Authorization': f"Bearer {token}"})
# ↑ Headers repeated every time, new TCP connection each time

# --- With Session: One connection, shared config ---
with requests.Session() as session:
    # Set default headers for ALL requests from this session
    session.headers.update({
        'Authorization': f"Bearer {token}",
        'Accept': 'application/vnd.github+json',
        'User-Agent': 'MyDeployScript/1.0',
    })

    # Now calls are cleaner and faster
    response1 = session.get(f"{base_url}/user")
    response2 = session.get(f"{base_url}/user/repos")
    response3 = session.get(f"{base_url}/rate_limit")
    # All three reuse the connection and share the headers

# --- GitHub API Client Using Session ---

class GitHubClient:
    """A simple GitHub API client using requests.Session."""

    BASE_URL = 'https://api.github.com'

    def __init__(self, token: str):
        self.session = requests.Session()
        self.session.headers.update({
            'Authorization': f"Bearer {token}",
            'Accept': 'application/vnd.github+json',
            'User-Agent': 'EngiDock-DevOps-Training/1.0',
        })

    def get(self, path: str, **kwargs):
        """Make a GET request to the GitHub API."""
        url = f"{self.BASE_URL}{path}"
        response = self.session.get(url, timeout=(3, 10), **kwargs)
        response.raise_for_status()
        return response.json()

    def post(self, path: str, payload: dict, **kwargs):
        """Make a POST request to the GitHub API."""

```

```

url = f"{self.BASE_URL}{path}"
response = self.session.post(url, json=payload, timeout=(3, 10), **kwargs)
response.raise_for_status()
return response.json()

def get_user(self) -> dict:
    return self.get('/user')

def get_repos(self) -> list:
    return self.get('/user/repos', params={'per_page': 100, 'sort': 'updated'})

def create_issue(self, owner: str, repo: str, title: str, body: str) -> dict:
    return self.post(
        f"/repos/{owner}/{repo}/issues",
        {'title': title, 'body': body}
    )

def get_rate_limit(self) -> dict:
    data = self.get('/rate_limit')
    core = data['rate']
    return {
        'limit': core['limit'],
        'remaining': core['remaining'],
        'reset_at': core['reset'],
    }

def close(self):
    self.session.close()

# Usage
token = os.environ.get('GITHUB_TOKEN')
if token:
    client = GitHubClient(token)
    try:
        user = client.get_user()
        print(f"Authenticated as: {user['login']}")
        rate_limit = client.get_rate_limit()
        print(f"Rate limit: {rate_limit['remaining']}/{rate_limit['limit']} remaining")
    finally:
        client.close()

```

Section 11 — Three Real DevOps Patterns

Pattern 1 — send_slack_alert()

```

import os
import sys
import requests
from datetime import datetime

def send_slack_alert(

```

```

webhook_url: str,
message: str,
channel: str = None,
username: str = 'Deploy Bot',
icon_emoji: str = ':rocket:',
color: str = None,
fields: list = None,
) -> bool:
    """
    Send a message to a Slack channel via incoming webhook.

    Args:
        webhook_url: The Slack webhook URL from environment
        message: The main message text (supports Slack markdown)
        channel: Override channel (e.g., '#alerts') - optional
        username: Display name for the bot
        icon_emoji: Bot icon (:rocket:, :warning:, :white_check_mark:)
        color: Attachment color ('good', 'warning', 'danger', or hex)
        fields: List of {'title': ..., 'value': ..., 'short': bool}

    Returns:
        True if message was sent successfully, False otherwise
    """
    # Build the payload
    payload = {
        'username': username,
        'icon_emoji': icon_emoji,
        'text': message,
    }

    if channel:
        payload['channel'] = channel

    # Add rich attachment if color or fields specified
    if color or fields:
        attachment = {
            'color': color or 'good',
            'text': message,
            'footer': f"EngiDock Deploy Bot | {datetime.utcnow().strftime('%Y-%m-%d %H:%M UTC')}}",
        }
        if fields:
            attachment['fields'] = fields

        payload['text'] = '' # Move text to attachment
        payload['attachments'] = [attachment]

    try:
        response = requests.post(
            webhook_url,
            json=payload,
            timeout=(3, 5),
        )

```

```

        response.raise_for_status()

        # Slack returns "ok" as the body text on success
        if response.text == 'ok':
            return True
        else:
            print(f"Slack responded unexpectedly: {response.text}", file=sys.stderr)
            return False

    except requests.exceptions.Timeout:
        print("Slack alert timed out – continuing anyway", file=sys.stderr)
        return False

    except requests.exceptions.RequestException as e:
        print(f"Slack alert failed: {e}", file=sys.stderr)
        return False

# Usage examples
SLACK_WEBHOOK = os.environ.get('SLACK_WEBHOOK_URL', 'https://httpbin.org/post')

# Simple message
send_slack_alert(SLACK_WEBHOOK, "Deploy started for myapp v2.1.0")

# Rich message with fields
send_slack_alert(
    webhook_url=SLACK_WEBHOOK,
    message="Deployment complete",
    icon_emoji=':white_check_mark:',
    color='good',
    fields=[
        {'title': 'Application', 'value': 'myapp', 'short': True},
        {'title': 'Version', 'value': '2.1.0', 'short': True},
        {'title': 'Environment', 'value': 'production', 'short': True},
        {'title': 'Duration', 'value': '2m 34s', 'short': True},
        {'title': 'Triggered by', 'value': 'github-actions', 'short': False},
    ]
)

# Failure alert
send_slack_alert(
    webhook_url=SLACK_WEBHOOK,
    message="Deploy FAILED for myapp",
    icon_emoji=':x:',
    color='danger',
    fields=[
        {'title': 'Stage', 'value': 'docker build', 'short': True},
        {'title': 'Error', 'value': 'Out of disk space', 'short': False},
    ]
)

```

Pattern 2 — call_with_retry() with Exponential Backoff

```
import time
import requests
import sys

def call_with_retry(
    url: str,
    method: str = 'GET',
    max_retries: int = 3,
    backoff: float = 2.0,
    retry_on: tuple = (429, 500, 502, 503, 504),
    timeout: tuple = (3, 10),
    **kwargs
) -> requests.Response:
    """
    Make an HTTP call with exponential backoff retry logic.

    Retries on:
        - Rate limiting (429) – respects Retry-After header
        - Server errors (500, 502, 503, 504)
        - Network errors (ConnectionError, Timeout)

    Does NOT retry on:
        - Client errors (400, 401, 403, 404, 422) – fix your code, not a transient error

    Args:
        url:          URL to call
        method:       HTTP method ('GET', 'POST', etc.)
        max_retries:  Maximum number of retry attempts
        backoff:      Base backoff multiplier (seconds). Doubles each retry.
        retry_on:     HTTP status codes to retry on
        timeout:      (connect_timeout, read_timeout) tuple
        **kwargs:     Passed to requests.request()

    Returns:
        requests.Response on success

    Raises:
        requests.exceptions.RequestException after all retries exhausted
    """
    last_exception = None

    for attempt in range(max_retries + 1):
        try:
            response = requests.request(method, url, timeout=timeout, **kwargs)

            # Success – return immediately
            if response.status_code not in retry_on:
                return response

            # 429: Rate limited – respect Retry-After header
```

```

        if response.status_code == 429:
            retry_after = int(response.headers.get('Retry-After', 60))
            print(f"Rate limited. Waiting {retry_after}s (Retry-After header)...",
                  file=sys.stderr)
            time.sleep(retry_after)
            continue

        # 5xx: Server error – exponential backoff
        wait = backoff * (2 ** attempt)
        print(f"Server error {response.status_code} on attempt {attempt +
1}/{max_retries + 1}. "
              f"Retrying in {wait:.0f}s...", file=sys.stderr)

        if attempt < max_retries:
            time.sleep(wait)

    except (requests.exceptions.ConnectionError, requests.exceptions.Timeout) as e:
        last_exception = e
        wait = backoff * (2 ** attempt)

        if attempt < max_retries:
            print(f"Network error on attempt {attempt + 1}/{max_retries + 1}: {e}. "
                  f"Retrying in {wait:.0f}s...", file=sys.stderr)
            time.sleep(wait)
        else:
            print(f"All {max_retries + 1} attempts failed. Last error: {e}",
                  file=sys.stderr)
            raise

    # If we exhausted retries on HTTP errors, return the last response
    print(f"All {max_retries + 1} attempts failed with HTTP errors", file=sys.stderr)
    return response

# Usage
try:
    response = call_with_retry(
        'https://api.example.com/deployments',
        method='POST',
        json={'version': '2.1.0'},
        headers={'Authorization': 'Bearer token123'},
        max_retries=3,
        backoff=2.0,
    )
    response.raise_for_status()
    print(f"Success: {response.json()}")

except requests.exceptions.RequestException as e:
    print(f"Failed after all retries: {e}", file=sys.stderr)
    sys.exit(1)

# Test with httpbin
response = call_with_retry('https://httpbin.org/status/200')
print(f"200 test: {response.status_code}")

```

```
# This will retry 3 times then succeed on the 4th call – test with httpbin
# (httpbin /status/500 always returns 500, so this will exhaust retries)
response = call_with_retry('https://httpbin.org/status/500', max_retries=2, backoff=1)
print(f"500 after retries: {response.status_code}")
```

Pattern 3 — check_endpoint_health()

```
import requests
import sys
import time
from datetime import datetime

def check_endpoint_health(
    url: str,
    timeout: float = 5,
    expected_status: int = 200,
    expected_key: str = None,
    expected_value: str = None,
) -> dict:
    """
    Check if an HTTP endpoint is healthy.

    Returns a dict:
    {
        'url':          str,
        'healthy':      bool,
        'status_code':  int,
        'latency_ms':   float,
        'reason':       str,
        'checked_at':   str,
    }
    """
    result = {
        'url':          url,
        'healthy':      False,
        'status_code':  0,
        'latency_ms':   0,
        'reason':       '',
        'checked_at':   datetime.utcnow().isoformat() + 'Z',
    }

    start = time.monotonic()

    try:
        response = requests.get(url, timeout=timeout)
        elapsed_ms = (time.monotonic() - start) * 1000

        result['status_code'] = response.status_code
        result['latency_ms'] = round(elapsed_ms, 2)

        # Check status code
```



```

        if response.status_code != expected_status:
            result['reason'] = f"Expected status {expected_status}, got {response.status_code}"
            return result

    # Optionally check a JSON field value
    if expected_key:
        try:
            data = response.json()
            actual = data.get(expected_key)
            if expected_value and str(actual) != str(expected_value):
                result['reason'] = f"Field '{expected_key}' = '{actual}', expected '{expected_value}'"
                return result
        except ValueError:
            result['reason'] = "Response is not valid JSON"
            return result

    result['healthy'] = True
    result['reason'] = f"OK ({elapsed_ms:.0f}ms)"
    return result

except requests.exceptions.Timeout:
    result['latency_ms'] = (time.monotonic() - start) * 1000
    result['reason'] = f"Timed out after {timeout}s"
    return result

except requests.exceptions.ConnectionError as e:
    result['latency_ms'] = (time.monotonic() - start) * 1000
    result['reason'] = f"Connection failed: {str(e)[:60]}"
    return result

except requests.exceptions.RequestException as e:
    result['latency_ms'] = (time.monotonic() - start) * 1000
    result['reason'] = f"Request error: {str(e)[:60]}"
    return result

def check_multiple_endpoints(endpoints: list) -> list:
    """
    Check health of multiple endpoints and return summary.
    endpoints: list of dicts with 'name', 'url', and optional 'timeout'
    """
    results = []
    for ep in endpoints:
        result = check_endpoint_health(
            url=ep['url'],
            timeout=ep.get('timeout', 5),
            expected_status=ep.get('expected_status', 200),
        )
        result['name'] = ep.get('name', ep['url'])
        results.append(result)
    return results

```

```

def print_health_report(results: list):
    """Print a formatted health check report."""
    healthy = [r for r in results if r['healthy']]
    unhealthy = [r for r in results if not r['healthy']]

    print("\n" + "=" * 65)
    print(f"  HEALTH CHECK REPORT - {datetime.utcnow().strftime('%Y-%m-%d %H:%M UTC')}")
    print("=" * 65)
    print(f"  {'Service':<25} {'Status':<10} {'Latency':>10} {'Detail'}")
    print(f"  {'-'*25} {'-'*10} {'-'*10} {'-'*15}")

    for r in results:
        status = "HEALTHY" if r['healthy'] else "UNHEALTHY"
        latency = f"{r['latency_ms']:.0f}ms" if r['latency_ms'] > 0 else "-"
        print(f"  {r['name']:<25} {status:<10} {latency:>10} {r['reason']}" )

    print("=" * 65)
    print(f"  Summary: {len(healthy)}/{len(results)} healthy")

    if unhealthy:
        print(f"\n  FAILING SERVICES:")
        for r in unhealthy:
            print(f"    - {r['name']}: {r['reason']}")

    return len(unhealthy) == 0  # True if all healthy

# Usage
endpoints = [
    {'name': 'httpbin-get',      'url': 'https://httpbin.org/get'},
    {'name': 'httpbin-ok',      'url': 'https://httpbin.org/status/200'},
    {'name': 'httpbin-broken',  'url': 'https://httpbin.org/status/503'},
    {'name': 'google-dns',      'url': 'https://dns.google/resolve?name=google.com&type=A'},
]

results = check_multiple_endpoints(endpoints)
all_healthy = print_health_report(results)

if not all_healthy:
    sys.exit(1)  # Non-zero exit: CI pipeline detects failure

```

Quick Reference Card

requests Library Quick Reference	
HTTP METHODS	
requests.get(url)	Retrieve data
requests.post(url, json={})	Create / send data
requests.put(url, json={})	Replace entire resource
requests.patch(url, json={})	Update part of resource
requests.delete(url)	Delete resource

requests.request('GET', url)	Generic – use for dynamic methods
COMMON PARAMETERS	
timeout=(3, 10)	(connect_timeout, read_timeout) in seconds
params={'key': 'value'}	Query parameters (?key=value)
json={'key': 'value'}	JSON body (sets Content-Type: application/json)
data={'key': 'value'}	Form body (sets Content-Type: form-urlencoded)
files={'file': open(..., 'rb')}	File upload (sets Content-Type: multipart)
headers={'Authorization': ...}	Custom request headers
auth=(user, pass)	HTTP Basic Auth
auth=HTTPBasicAuth(u, p)	HTTP Basic Auth (explicit)
RESPONSE OBJECT	
response.status_code	200, 404, 500, etc.
response.ok	True if status < 400
response.reason	'OK', 'Not Found', etc.
response.text	Response body as string
response.content	Response body as bytes
response.json()	Parse body as JSON → dict
response.headers	Case-insensitive dict of response headers
response.headers.get('k')	Safe header read (None if missing)
response.url	Final URL after redirects
response.elapsed	timedelta of request duration
response.raise_for_status()	Raise HTTPError on 4xx/5xx
EXCEPTIONS	
HTTPError	4xx/5xx status (after raise_for_status)
ConnectionError	DNS, refused connection, network down
Timeout	Exceeded connect or read timeout
ConnectTimeout	Couldn't establish connection
ReadTimeout	Server stopped responding
TooManyRedirects	Redirect loop
RequestException	Catch-all for all requests errors
SESSION	
with requests.Session() as s:	Context manager – auto-closes
s.headers.update({...})	Set default headers for all session calls
s.auth = (user, pass)	Set default auth for all session calls
s.get(url) / s.post(url)	Same as requests.get but reuses connection
AUTHENTICATION PATTERNS	
Bearer token	headers={'Authorization': f"Bearer {token}"}
Basic auth	auth=(username, password)
API key in header	headers={'X-API-Key': api_key}
API key in query param	params={'api_key': api_key}
STATUS CODE CHEAT SHEET	
200 OK	Success with body
201 Created	POST created new resource
204 No Content	Success, no body (common for DELETE)
400 Bad Request	Your payload is malformed
401 Unauthorized	Authentication required or invalid
403 Forbidden	Authenticated but not allowed

404 Not Found	Resource doesn't exist	
429 Too Many Requests	Rate limited – check Retry-After header	
500 Internal Server Error	API bug	
502 Bad Gateway	Proxy/load balancer issue	
503 Service Unavailable	API is down, retry later	
504 Gateway Timeout	Upstream service timed out	

SECURITY RULES FOR API CALLS

- NEVER hardcode tokens, keys, or passwords in source code
- ALWAYS read credentials from environment variables
- ALWAYS set a timeout – no exception
- ALWAYS check status before parsing response body
- ALWAYS handle exceptions – network calls can always fail
- Use `raise_for_status()` for simple scripts; catch specific errors for production

Hands-On Exercises

Exercise 1 — Exploring GET Requests and Response Headers

```
#!/usr/bin/env python3
"""
exercisel_get_requests.py – Explore GET requests with httpbin.org.

Run: python exercisel_get_requests.py
"""

import requests
import sys

def part1_basic_get():
    """GET https://httpbin.org/get – inspect status, headers, body."""
    print("=" * 55)
    print("Part 1: Basic GET Request")
    print("=" * 55)

    response = requests.get('https://httpbin.org/get', timeout=(3, 10))

    # 1. Status code
    print(f"\nStatus code: {response.status_code}")
    print(f"Status reason: {response.reason}")
    print(f"Success: {response.ok}")
    print(f"Latency: {response.elapsed.total_seconds() * 1000:.0f}ms")

    # 2. Three interesting response headers
    print(f"\nSelected Response Headers:")
    interesting_headers = [
        'Content-Type',
        'Content-Length',
        'Server',
        'X-Powered-By',
    ]
```

```

        'Access-Control-Allow-Origin',
    ]
    for header in interesting_headers:
        value = response.headers.get(header, '(not present)')
        print(f" {header:<35} {value}")

# 3. Parse JSON and print 'origin' field
data = response.json()
print(f"\nJSON body - 'origin' field: {data['origin']}")
print(f"JSON body - 'url' field: {data['url']}")
print(f"JSON body - 'headers' sent: {list(data['headers'].keys())}")

def part2_get_with_params():
    """GET with query parameters."""
    print("\n" + "=" * 55)
    print("Part 2: GET with Query Parameters")
    print("=" * 55)

    params = {
        'name': 'devops',
        'day': '48',
        'tool': 'requests',
        'env': 'production',
    }

    response = requests.get(
        'https://httpbin.org/get',
        params=params,
        timeout=(3, 10)
    )

    print(f"\nURL with params: {response.url}")

    data = response.json()
    print(f"\nQuery params received by server:")
    for key, value in data['args'].items():
        print(f" {key} = {value}")

# Demonstrate that requests handles special characters
special_params = {
    'message': 'hello world', # Space
    'filter': 'status=active', # Equals sign
    'tags': 'web,api,v2', # Comma
}

response2 = requests.get(
    'https://httpbin.org/get',
    params=special_params,
    timeout=(3, 10)
)

print(f"\nURL with special chars (auto-encoded):")
print(f" {response2.url}")

```

```

data2 = response2.json()
print(f"\nDecoded params on server side:")
for key, value in data2['args'].items():
    print(f"  {key} = {value}")

def part3_custom_headers():
    """GET with custom headers."""
    print("\n" + "=" * 55)
    print("Part 3: GET with Custom Headers")
    print("=" * 55)

    headers = {
        'User-Agent':      'EngiDock-Day48-Exercise/1.0',
        'X-Deploy-Env':    'staging',
        'X-Build-Version':  '2.1.0',
        'Accept':          'application/json',
    }

    response = requests.get(
        'https://httpbin.org/headers',
        headers=headers,
        timeout=(3, 10)
    )

    data = response.json()
    print(f"\nHeaders seen by the server:")
    for key, value in sorted(data['headers'].items()):
        marker = " ← (our custom header)" if key.startswith('X-') or key == 'User-Agent'
    else ""
        print(f"  {key}: {value}{marker}")

def main():
    try:
        part1_basic_get()
        part2_get_with_params()
        part3_custom_headers()
        print("\nAll GET exercises completed successfully.")
        sys.exit(0)
    except requests.exceptions.RequestException as e:
        print(f"\nERROR: Request failed: {e}", file=sys.stderr)
        print("      Check your internet connection and try again.", file=sys.stderr)
        sys.exit(1)

if __name__ == '__main__':
    main()

```

Exercise 2 — POST: json= vs data= — Understanding the Difference

```
#!/usr/bin/env python3
"""
exercise2_post_comparison.py – Understand json= vs data= in POST requests.

Run: python exercise2_post_comparison.py
"""

import requests
import sys
import json

def post_with_json():
    """POST with json= – sends application/json body."""
    print("=" * 60)
    print("POST with json= (Content-Type: application/json)")
    print("=" * 60)

    payload = {
        'app': 'tracker',
        'version': '1.0',
        'env': 'production',
        'tags': ['web', 'api'], # Nested structures work
        'metrics': {'uptime': 99.9}, # Also nested
    }

    response = requests.post(
        'https://httpbin.org/post',
        json=payload,
        timeout=(3, 10)
    )

    data = response.json()

    print(f"\nContent-Type sent: {data['headers']['Content-Type']}")
    print(f"\nwhat server received as JSON:")
    print(f"  {json.dumps(data['json'], indent=4)}")
    print(f"\nwhat server received as FORM (should be empty): {data['form']}")
    print(f"\nKey point: json= sends nested objects, arrays, and all JSON types.")

def post_with_data():
    """POST with data= – sends application/x-www-form-urlencoded body."""
    print("\n" + "=" * 60)
    print("POST with data= (Content-Type: application/x-www-form-urlencoded)")
    print("=" * 60)

    form_data = {
        'app': 'tracker',
        'version': '1.0',
        'env': 'production',
        # Note: you CANNOT send nested objects or arrays with data=
    }
```

```

}

response = requests.post(
    'https://httpbin.org/post',
    data=form_data,
    timeout=(3, 10)
)

data = response.json()

print(f"\nContent-Type sent: {data['headers']['Content-Type']}")
print(f"\nwhat server received as FORM:")
for key, value in data['form'].items():
    print(f"    {key}: {value}")
print(f"\nwhat server received as JSON (should be None): {data['json']}")
print(f"\nKey point: data= sends flat key=value pairs. No nesting.")

def post_with_raw_json_string():
    """POST with raw JSON string – shows what json= does internally."""
    print("\n" + "=" * 60)
    print("POST with raw JSON string (manually, to show what json= does)")
    print("=" * 60)

    payload = {'app': 'tracker', 'version': '1.0'}
    json_string = json.dumps(payload)

    response = requests.post(
        'https://httpbin.org/post',
        data=json_string.encode('utf-8'),
        headers={'Content-Type': 'application/json'}, # Must set manually
        timeout=(3, 10)
    )

    data = response.json()
    print(f"\nContent-Type sent: {data['headers']['Content-Type']}")
    print(f"JSON received: {data['json']}")
    print(f"\nThis is exactly what json=payload does automatically.")

def comparison_summary():
    """Print the decision guide."""
    print("\n" + "=" * 60)
    print("DECISION GUIDE: When to Use Which")
    print("=" * 60)
    print("""
json=payload          → Use when calling REST APIs, webhooks (Slack, Github,
                        PagerDuty), any modern API that returns JSON.
                        Sets Content-Type: application/json automatically.
                        Supports nested objects and arrays.

data=payload          → Use when submitting HTML forms, calling legacy APIs,
                        OAuth token endpoints, some authentication flows.
                        Sets Content-Type: application/x-www-form-urlencoded.
                        Flat key=value only.
    """)

```



```
files=files          → Use when uploading files (log files, artifacts, images).
                        Sets Content-Type: multipart/form-data automatically.
```

How to choose:

1. Read the API documentation – it will tell you what Content-Type it expects
2. If the API response is JSON → use json=
3. If the API says "send as form" → use data=
4. If you're uploading a file → use files=

```
""")
```

```
def main():
    try:
        post_with_json()
        post_with_data()
        post_with_raw_json_string()
        comparison_summary()
        print("Exercise 2 complete.")
        sys.exit(0)
    except requests.exceptions.RequestException as e:
        print(f"\nERROR: {e}", file=sys.stderr)
        sys.exit(1)

if __name__ == '__main__':
    main()
```

Exercise 3 — Error Handling — get_safe() That Never Raises

```
#!/usr/bin/env python3
```

```
"""
```

exercise3_safe_requests.py – Write a get_safe() function that never raises.

Run: python exercise3_safe_requests.py

```
"""
```

```
import requests
```

```
import sys
```

```
from typing import Optional, Tuple
```

```
def get_safe(url: str, timeout: tuple = (3, 10)) -> Tuple[Optional[dict], Optional[str]]:
    """
```

Make a GET request that NEVER raises an exception.

Returns:

- (data, None) – on success: data is the parsed JSON (or response text)
- (None, error) – on failure: error is a human-readable error message

This pattern is useful when:

- You want to handle all failure cases in the caller
- You're checking multiple endpoints and want to continue even if some fail
- You're building a health check that must always return a result

```

The caller checks: if error: ... else: use data
"""
try:
    response = requests.get(url, timeout=timeout)

    # Separate handling for different status code ranges
    if response.status_code == 200:
        # Try to parse as JSON, fall back to text
        try:
            return response.json(), None
        except ValueError:
            return response.text, None

    elif response.status_code == 404:
        return None, f"404 Not Found – {url} does not exist"

    elif response.status_code == 401:
        return None, f"401 Unauthorized – authentication required"

    elif response.status_code == 403:
        return None, f"403 Forbidden – you don't have permission"

    elif response.status_code == 429:
        retry_after = response.headers.get('Retry-After', '60')
        return None, f"429 Rate Limited – retry after {retry_after}s"

    elif 500 <= response.status_code < 600:
        return None, f"{response.status_code} Server Error – the API is broken"

    else:
        return None, f"Unexpected status {response.status_code}: {response.reason}"

except requests.exceptions.ConnectTimeout:
    return None, f"Connection timeout – couldn't reach {url} in {timeout[0]}s"

except requests.exceptions.ReadTimeout:
    return None, f"Read timeout – server at {url} connected but didn't respond in {timeout[1]}s"

except requests.exceptions.ConnectionError:
    # Extract just the useful part of the error message
    return None, f"Connection error – check if {url} is reachable"

except requests.exceptions.TooManyRedirects:
    return None, f"Too many redirects – {url} is looping"

except requests.exceptions.RequestException as e:
    return None, f"Request error: {str(e)[:100]}"

def test_get_safe():
    """Test get_safe with various status codes."""
    print("Testing get_safe() function")

```

```

print("=" * 60)

test_cases = [
    ('https://httpbin.org/status/200', "Should succeed"),
    ('https://httpbin.org/get', "Should return JSON"),
    ('https://httpbin.org/status/404', "Should return 404 error"),
    ('https://httpbin.org/status/500', "Should return 500 error"),
    ('https://httpbin.org/status/429', "Should return rate limit error"),
    ('https://httpbin.org/delay/20', "Should timeout (read timeout)"),
    ('https://this-domain-does-not-exist-99999.com/', "Should fail DNS"),
]

for url, description in test_cases:
    print(f"\n Test: {description}")
    print(f" URL: {url}")

    data, error = get_safe(url, timeout=(3, 5))

    if error:
        print(f" Result: FAILED - {error}")
    else:
        # Show just a snippet of data
        if isinstance(data, dict):
            keys = list(data.keys())[:3]
            print(f" Result: SUCCESS - JSON with keys: {keys}")
        else:
            print(f" Result: SUCCESS - text ({len(str(data))} chars)")

def batch_health_check(urls: list) -> dict:
    """
    Check multiple URLs using get_safe. Never fails overall.
    Returns summary of healthy vs failed endpoints.
    """
    results = {'healthy': [], 'failed': []}

    for url in urls:
        data, error = get_safe(url)
        if error:
            results['failed'].append({'url': url, 'error': error})
        else:
            results['healthy'].append({'url': url})

    return results

def main():
    test_get_safe()

    print("\n" + "=" * 60)
    print("Batch health check using get_safe:")
    print("=" * 60)

    urls_to_check = [
        'https://httpbin.org/get',
    ]

```

```

    'https://httpbin.org/status/200',
    'https://httpbin.org/status/503',
    'https://dns.google/resolve?name=github.com&type=A',
]

results = batch_health_check(urls_to_check)

print(f"\n Healthy ({len(results['healthy'])}):")
for r in results['healthy']:
    print(f"    OK {r['url']}")

print(f"\n Failed ({len(results['failed'])}):")
for r in results['failed']:
    print(f"    ERR {r['url']}")
    print(f"        {r['error']}")

print(f"\nget_safe() never raised an exception – even on errors and timeouts.")
sys.exit(0)

if __name__ == '__main__':
    main()

```

Exercise 4 — send_slack_message() with Real or Simulated Webhook

```

#!/usr/bin/env python3
"""
exercise4_slack_webhook.py – Send a Slack message via webhook.

Run with a real webhook:
    SLACK_WEBHOOK_URL=https://hooks.slack.com/services/... python exercise4_slack_webhook.py

Run in simulation mode (no Slack account needed):
    python exercise4_slack_webhook.py
    (Uses https://httpbin.org/post to simulate the Slack endpoint)
"""

import os
import sys
import json
import requests
from datetime import datetime

def send_slack_message(
    webhook_url: str,
    text: str,
    username: str = 'DevOps Bot',
    icon_emoji: str = ':robot_face:',
) -> tuple:
    """
    Send a plain text message to Slack via incoming webhook.

```

Args:

webhook_url: Slack webhook URL (or httpbin.org/post for testing)
text: Message text (Slack markdown supported)
username: Display name shown in Slack
icon_emoji: Emoji shown as the bot's avatar

Returns:

(True, None) on success
(False, error_message) on failure

"""

payload = {

'text': text,
 'username': username,
 'icon_emoji': icon_emoji,

}

try:

response = requests.post(
 webhook_url,
 json=payload,
 timeout=(3, 5),
)

Real Slack webhook returns "ok" as plain text on success

httpbin.org returns a JSON echo of what we sent

if 'hooks.slack.com' in webhook_url:

if response.status_code == 200 and response.text == 'ok':
 return True, None

else:

return False, f"Slack returned: {response.status_code} - {response.text}"

else:

Simulation mode with httpbin

response.raise_for_status()

return True, None

except requests.exceptions.Timeout:

return False, "Webhook request timed out"

except requests.exceptions.ConnectionError:

return False, "Could not connect to webhook URL"

except requests.exceptions.RequestException as e:

return False, str(e)

def send_slack_deploy_notification(
 webhook_url: str,

app_name: str,

version: str,

environment: str,

status: str, # 'started', 'success', 'failure'

details: str = '',

) -> tuple:

"""

Send a structured deployment notification to Slack.

Args:

webhook_url: Slack webhook URL
app_name: Name of the application being deployed
version: Version/tag being deployed
environment: Target environment (staging, production, etc.)
status: 'started', 'success', or 'failure'
details: Additional context (error message, duration, etc.)

Returns:

(True, None) on success, (False, error) on failure

"""

```
STATUS_CONFIG = {
    'started': {
        'emoji': ':arrows_counterclockwise:',
        'prefix': 'Deploy Started',
        'color': '#439FE0',
    },
    'success': {
        'emoji': ':white_check_mark:',
        'prefix': 'Deploy Successful',
        'color': 'good',
    },
    'failure': {
        'emoji': ':x:',
        'prefix': 'Deploy FAILED',
        'color': 'danger',
    },
}

config = STATUS_CONFIG.get(status, STATUS_CONFIG['started'])
timestamp = datetime.utcnow().strftime('%Y-%m-%d %H:%M UTC')

# Build the Slack message payload
payload = {
    'username': 'Deploy Bot',
    'icon_emoji': config['emoji'],
    'text': f"{config['emoji']} {config['prefix']}* - {app_name}",
    'attachments': [
        {
            'color': config['color'],
            'fields': [
                {'title': 'Application', 'value': app_name, 'short': True},
                {'title': 'Version', 'value': version, 'short': True},
                {'title': 'Environment', 'value': environment, 'short': True},
                {'title': 'Timestamp', 'value': timestamp, 'short': True},
            ],
            'footer': 'EngiDock CI/CD Pipeline',
        }
    ],
}
```

```

# Add details field if provided
if details:
    payload['attachments'][0]['fields'].append({
        'title': 'Details',
        'value': details,
        'short': False,
    })

try:
    response = requests.post(
        webhook_url,
        json=payload,
        timeout=(3, 5),
    )
    response.raise_for_status()
    return True, None

except requests.exceptions.Timeout:
    return False, "webhook timed out"
except requests.exceptions.RequestException as e:
    return False, str(e)

def simulate_and_print(webhook_url: str, payload_description: str, actual_payload: dict):
    """Show what payload would be sent to Slack."""
    print(f"\n Payload ({payload_description}):")
    print(f"  " + "-" * 50)
    print("  " + json.dumps(actual_payload, indent=4).replace('\n', '\n  '))

def main():
    # Get webhook URL from environment or use httpbin simulation
    webhook_url = os.environ.get('SLACK_WEBHOOK_URL')
    using_real_slack = webhook_url is not None

    if not using_real_slack:
        webhook_url = 'https://httpbin.org/post'
        print("No SLACK_WEBHOOK_URL set – running in simulation mode.")
        print("Using https://httpbin.org/post to echo the payload.\n")
        print("To use a real Slack webhook:")
        print("  export SLACK_WEBHOOK_URL=https://hooks.slack.com/services/T.../B.../xxx\n")
    else:
        print(f"Using real Slack webhook: {webhook_url[:50]}...\n")

    print("=" * 60)
    print("Exercise 4: Slack webhook Sender")
    print("=" * 60)

    # --- Test 1: Simple message ---
    print("\nTest 1: Simple text message")
    success, error = send_slack_message(
        webhook_url=webhook_url,
        text="Hello from Day 48! EngiDock DevOps training is awesome.",
        username='Day 48 Bot',
        icon_emoji=':mortar_board:',
    )

```

```

)

if success:
    print(" Result: SENT successfully")
    if not using_real_slack:
        print(" (check httpbin.org response - it echoed what we POSTed)")
else:
    print(f" Result: FAILED - {error}")

# --- Test 2: Deploy started ---
print("\nTest 2: Deploy started notification")
success, error = send_slack_deploy_notification(
    webhook_url=webhook_url,
    app_name='myapp',
    version='v2.1.0',
    environment='staging',
    status='started',
    details='Triggered by: github-actions | Branch: main | Commit: a3f9b12',
)
print(f" Result: {'SENT' if success else 'FAILED - ' + error}")

# --- Test 3: Deploy success ---
print("\nTest 3: Deploy success notification")
success, error = send_slack_deploy_notification(
    webhook_url=webhook_url,
    app_name='myapp',
    version='v2.1.0',
    environment='staging',
    status='success',
    details='Duration: 3m 22s | 3 containers healthy | Health check passed',
)
print(f" Result: {'SENT' if success else 'FAILED - ' + error}")

# --- Test 4: Deploy failure ---
print("\nTest 4: Deploy failure notification")
success, error = send_slack_deploy_notification(
    webhook_url=webhook_url,
    app_name='myapp',
    version='v2.1.0',
    environment='staging',
    status='failure',
    details='Stage: docker build | Error: no space left on device | Exit code: 1',
)
print(f" Result: {'SENT' if success else 'FAILED - ' + error}")

# --- Show what the payload looks like ---
print("\n" + "=" * 60)
print("Payload Structure (what gets POSTed to the webhook):")
print("=" * 60)

example_payload = {
    'username': 'Deploy Bot',
    'icon_emoji': ':white_check_mark:',

```



```

        'text': ':white_check_mark: *Deploy Successful* - myapp',
        'attachments': [{
            'color': 'good',
            'fields': [
                {'title': 'Application', 'value': 'myapp', 'short': True},
                {'title': 'Version', 'value': 'v2.1.0', 'short': True},
                {'title': 'Environment', 'value': 'staging', 'short': True},
                {'title': 'Timestamp', 'value': '2026-05-19 10:00 UTC', 'short': True},
            ],
            'footer': 'EngiDock CI/CD Pipeline',
        }],
    }

    print(json.dumps(example_payload, indent=2))

    if using_real_slack:
        print("\nCheck your Slack channel – messages should have appeared.")
    else:
        print("\nsimulation complete. Set SLACK_WEBHOOK_URL to send real messages.")

    sys.exit(0)

if __name__ == '__main__':
    main()

```

What's Next — Day 49: Health Check Script — Putting requests to Operational Use

Tomorrow you'll build a complete, production-ready health check script that combines everything from today with everything from Day 47. The script will:

- Read a list of endpoints from a YAML configuration file
- Check each endpoint with the right timeout and expected status code
- Retry failed endpoints before marking them down
- Send a Slack alert if any endpoints are unhealthy
- Write results to a JSON log file for trend analysis
- Exit with code 0 if all healthy, code 1 if any are failing

This is a real script that DevOps engineers use in production. You'll run it on a cron job (Day 22) and combine it with the monitoring knowledge from later weeks. By Day 49, you'll have written three complete scripts that you can actually use: `system_info.py`, `health_check.py`, and a template for any API integration your team needs.

The pattern is always the same: read config from environment, make HTTP calls safely, handle errors explicitly, exit with the right code, log what happened.

